

3507. Minimum Pair Removal to Sort Array I

PLATFORM

Platform: LeetCode

DIFFICULTY

EASY

DATE

Solved: July 7, 2026

Problem Summary

Given an integer array `nums`, repeatedly select the adjacent pair with the minimum sum (leftmost if tie), replace them with their sum, and return the number of operations needed to make the array non-decreasing.

Approach

Simulate the process directly. Since constraints are tiny (≤ 50 elements), brute-force each step — check if done, find the minimum-sum adjacent pair, merge them, repeat. Use an `ArrayList<Long>` to allow dynamic removal and avoid overflow.

Key Insights

- Simulation is valid here **only because** constraints are tiny (≤ 50 elements) — don't use this on large inputs.
- `ArrayList` is needed over plain array because the list shrinks after each merge — fixed arrays can't do this cleanly.
- Use `Long` to prevent integer overflow when summing pairs repeatedly.
- `isNonDecreasing()` helper keeps the main loop clean and readable.

Observations

- Each operation reduces array size by exactly 1 $\rightarrow$ at most $n-1$ operations total.
- The "leftmost if tie" rule is handled naturally by using `strict <` (not `<=`) when updating `minSum`.
- For the harder version (LC 3510, same problem but n up to 10^5), this simulation would TLE — needs a priority queue + linked list.

Points to Remember

1. **Simulation pattern:** when constraints are small, simulate exactly what the problem says — no tricks needed.
2. `list.set(i, val) + list.remove(i+1)` is the clean way to merge two adjacent elements in a List.
3. Always reach for `Long` when values accumulate through repeated addition — `int` silently overflows.
4. Strict `<` for leftmost-tie rule; `<=` would incorrectly prefer rightmost.
5. This problem is a warm-up for the heap + doubly-linked-list pattern used in LC 3510 (same problem, large constraints).

Algorithm

Load `nums` into `ArrayList<Long>`.

While array is not non-decreasing:

- Scan all adjacent pairs, track index of minimum sum (leftmost if tie).
- `arr.set(idx, minSum)` — replace left element with sum.
- `arr.remove(idx + 1)` — delete right element.
- Increment ops.

Return ops.

Time Complexity & Space Complexity

$O(n^2)$ — up to n merges, each requiring an $O(n)$ scan for minimum pair.

$O(n)$ — `ArrayList` copy of the input array.

Linked List

DescriptionEditorialSolutionsSubmissions

3507. Minimum Pair Removal to Sort Array I

Solved

EasyTopicsCompaniesHint

Given an array `nums`, you can perform the following operation any number of times:

- Select the **adjacent** pair with the **minimum** sum in `nums`. If multiple such pairs exist, choose the leftmost one.
- Replace the pair with their sum.

Return the **minimum number of operations** needed to make the array **non-decreasing**.

An array is said to be **non-decreasing** if each element is greater than or equal to its previous element (if it exists).

Example 1:

Input: `nums = [5,2,3,1]`

Output: 2

Explanation:

- The pair `(3,1)` has the minimum sum of 4. After replacement, `nums = [5,2,4]`.
- The pair `(2,4)` has the minimum sum of 6. After replacement, `nums = [5,6]`.

The array `nums` became non-decreasing in two operations.

Example 2:

Input: `nums = [1,2,2]`

Output: 0

Explanation:

Accepted806 / 806 testcases passed

Sahil Khan submitted at Jul 06, 2026 18:38

Runtime

3 ms | Beats 69.27%

Memory

44.73 MB | Beats 14.59%



Code | Java

```
1 class Solution {
2     public int minimumPairRemoval(int[] nums) {
3         List<Long> arr = new ArrayList<>();
4         for (int n : nums) arr.add((long) n);
5         int ops = 0;
6
7         while (!isNonDecreasing(arr)) {
8             int idx = 0;
9             long minSum = arr.get(0) + arr.get(1);
10            for (int i = 1; i < arr.size() - 1; i++) {
11                long sum = arr.get(i) + arr.get(i + 1);
12                if (sum < minSum) {
13                    minSum = sum;
14                    idx = i;
15                }
16            }
17            arr.set(idx, minSum); // replace left with sum
18            arr.remove(idx + 1); // delete right element
19            ops++;
20        }
21        return ops;
22    }
23
24    private boolean isNonDecreasing(List<Long> arr) {
25        for (int i = 0; i < arr.size() - 1; i++) {
26            if (arr.get(i) > arr.get(i + 1)) return false;
27        }
28        return true;
29    }
30 }
```

Write your notes here

Code

JavaAuto

```
1 class Solution {
2     public int minimumPairRemoval(int[] nums) {
3         List<Long> arr = new ArrayList<>();
4         for (int n : nums) arr.add((long) n);
5         int ops = 0;
6
7         while (!isNonDecreasing(arr)) {
8             int idx = 0;
9             long minSum = arr.get(0) + arr.get(1);
10            for (int i = 1; i < arr.size() - 1; i++) {
11                long sum = arr.get(i) + arr.get(i + 1);
12                if (sum < minSum) {
13                    minSum = sum;
14                    idx = i;
15                }
16            }
17            arr.set(idx, minSum); // replace left with sum
18            arr.remove(idx + 1); // delete right element
19            ops++;
20        }
21        return ops;
22    }
23
24    private boolean isNonDecreasing(List<Long> arr) {
25        for (int i = 0; i < arr.size() - 1; i++) {
26            if (arr.get(i) > arr.get(i + 1)) return false;
27        }
28        return true;
29    }
30 }
```

SavedLn 1, Col 1

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2